小测 1

An integer constant consisting of a sequence of digits is taken to be decimal (base ten) unless it begins with **0** (digit zero). A sequence of digits starting with **0** is taken to be an octal integer (base eight). The digits **8** and **9** are not octal digits. A sequence of digits preceded by **0x** or **0X** is taken to be a hexadecimal integer (base sixteen). The hexadecimal digits include **a** or **A** through **f** or **F**…. The type of an integer depends on its … suffix. [An integer may have no suffix.] …If [the integer] if suffixed by **u** or **U**, its type is …. If it is suffixed by **l** or **L**, its type is …. [Either or both suffixes may appear at most once each in either order.]

试为 C++语言的整型常量设计一个正规定义式，要求其中仅使用正则表达式的基本算子“|”、“*”和连接，禁止使用 lex 或 flex 等软件工具扩展的表示；在正则表达式中可以使用括号和空串ε。

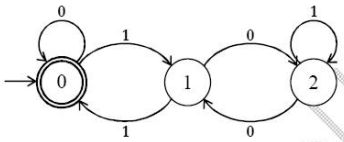
〖参考答案〗

Octal → **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7**
NonZero → **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9**
Decimal → **0** | NonZero
Hex → Decimal | **a** | **A** | **b** | **B** | **c** | **C** | **d** | **D** | **e** | **E** | **f** | **F**
OptU → **u** | **U** | ε
OptL → **l** | **L** | ε
Suffix → OptU OptL | OptL OptU
Number → **0** Octal* | NonZero Decimal* | (**0x** | **0X**) Hex Hex*
Integer → Number Suffix

构造一个 DFA，该 DFA 识别的语言是所有能被 3 整除的二进制数。

〖参考答案〗

解题关键：用 3 个状态分别表示输入串整除 3 的余数为 0、1 和 2；状态转移 $\delta(0, 1) = 1$ 相当于 $y = x * 2 + 1$，即 x 被 3 整除则有 y 的余数为 1。

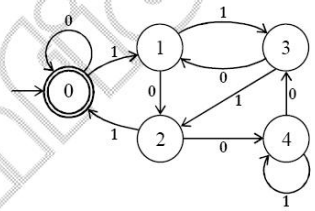


【备注】

1、注意题目要求画出 DFA，如果画成 NFA 则属基本概念理解不清。

2、识别被 3 整除后余 1 或余 2 的数：修改终态即可。

3、被 7 整除的数：状态数目变为 7。类似地，识别能被 5 整除的所有二进制数的 DFA 如下：



4、八进制或十进制数：每一状态的射出弧分别有 8 条或 10 条。

Confidential

小测 2

1、画出一个 DFA，该 DFA 恰好识别所有不含 **011** 子串的所有二进制串（注意不要 *under/over spec.*，下同）。

2、再画一个 DFA，该 DFA 恰好识别所有不含 **101** 子串的所有二进制串。

3、从上述两个例子中，总结出解决这一类问题的普遍规律。

小测 3

考虑两个 LL(1)文法，每个都只有一个非终结符号（设第一个文法的非终结符号为 A，第二个文法的非终结符号为 B），且不含ε产生式。考察第三个文法，该文法的开始符号为 S，并且产生式通过合并上述两个文法的所有产生式得到，然后再添加以下产生式：

S → A | B

请回答以下问题并解释理由：

1 在第三个文法中，是否可能从文法的开始符号 S 推导出空串？为什么？

2 为保证第三个文法是 LL(1)的，原先的两个文法必须满足的充分必要条件是什么？请不要给出 A 或 B 的具体例子，你需要描述的是针对上述形式的所有文法均须满足的充要条件。

3 为保证第三个文法是无二义的，原先的两个文法必须满足的充分必要条件是什么？请不要给出 A 或 B 的具体例子，你需要描述的是针对上述形式的所有文法均须满足的充要条件。

〖参考答案〗

1 不可能，因为从 A 或 B 均不可能推导出空串。

2 充要条件：A 和 B 能够推出的首字符不可相同，即 FIRST(A) ⋂ FIRST(B) = ∅。注意，题目已指出这两个文法是 LL(1)的，且 A 或 B 均不可能推导出空串。

3 充要条件：这两个文法产生的语言的交集为空，即 L(A) ⋂ L(B) = ∅，其中 L(X)表示由文法 X 产生的语言。注意，由于题目已指出这两个文法是 LL(1)的，故它们一定是无二义的。

考虑下面文法 G，

S → Q **a**

Q → Q **b** | **c** | **ε**

1 先消除文法中的直接左递归，得到新文法 G'。

2 再计算新文法 G'中所有非终结符（Nonterminal）的 FIRST 集和 FOLLOW 集。

3 最后构造 G'的 LL(1)分析表。

【参考答案】

1 消除左递归，得到文法 G'：

S → Q **a**

Q → **c** Q' | Q'

Q' → **b** Q' | **ε**

2 求非终结符的 FIRST 集和 FOLLOW 集：

FIRST(S) = { **a**, **b**, **c** }, FIRST(Q) = { **b**, **c**, **ε** }, FIRST(Q') = { **b**, **ε** }

FOLLOW(S) = { **$** }, FOLLOW(Q) = FOLLOW(Q') = { **a** }

3 最后构造 LL(1)分析表：

	a	**b**	**c**	**$**
S	Q **a**	Q **a**	Q **a**	
Q	Q'	Q'	**c** Q'	
Q'	**ε**	**b** Q'		

小测 4

考虑如下的上下文无关文法（Context-Free Grammar）：

```
S  →  A
A  →  A + A  |  B + +
B  →  y
```

其中，每一个“+”是一个单独的符号（即单词，token）。由于开始符号（Start Symbol）仅有一个产生式，故该文法无需进行拓广（augment），S可直接作为LR分析器的开始符号。

1、 画出输入串 **y + + + y + +** 的分析树（Parse Tree）。

2、 记LR(1)有效项目（Valid Item）I = [S → • A, $]，求closure({I})。提示：即求识别该文法所有活前缀（Viable Prefix）的DFA的开始状态中包含的所有LR(1)有效项目。

3、 在构造识别该文法的所有活前缀的确定有限自动机（DFA）时，有一个状态 Q = {

[A → A + A •, + / $],

[A → A • + A, + / $]

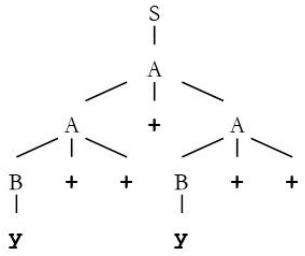
}，求 go(Q, +)。提示：即求从状态Q出发，经过有向边+到达的状态；列出该状态中的所有LR(1)有效项目。

4、 该文法存在“移进－归约”冲突（Shift-Reduce Conflict）。请给出上述DFA中存在这种冲突的状态（即列出状态中所有的LR(1)有效项目），并指出在处理哪一个输入符号时会发生冲突?

□

【参考答案】

1、分析树如下：



2、closure({ [S → • A, $] }) = {

```
[S → • A, $],
[A → • A + A, + / $],
[A → • B + +, + / $],
[B → • y, +] }
```

3、go(Q, +) = closure({ [A → A + • A, + / $] }) = {

```
[A → A + • A, + / $],
[A → • A + A, + / $],
[A → • B + +, + / $],
[B → • y, +] }
```

4、在包含项目[A → A + A •, + / $]和[A → A • + A, + / $]的那个状态、遇到输入符号+时，会发生“移进－归约”冲突。

小测 5

Item #5.1 给定文法

```
S → B
B → BE+ | −E
E → e | Ee | ε
```

(1) 写出该文法的拓广文法，并为新文法的每一产生式加上编号（第一个产生式从 0 开始编号）；再用一句话解释在构造 LR 分析表时拓广文法的原因。

(2) 求出所有非终结符号的 FIRST 集以备后用。

(3) 根据 LR(1)有效项目集族画出识别该文法所有活前缀的 DFA。

(4) 判断该文法是否一个 LR(1)文法，并说明你的理由。

Item #5.1 〖参考答案〗

(1) 拓广文法：

```
(0)  S' → S
(1)  S → B
(2)  B → BE+
(3)  B → −E
(4)  E → e
(5)  E → Ee
(6)  E → ε
```

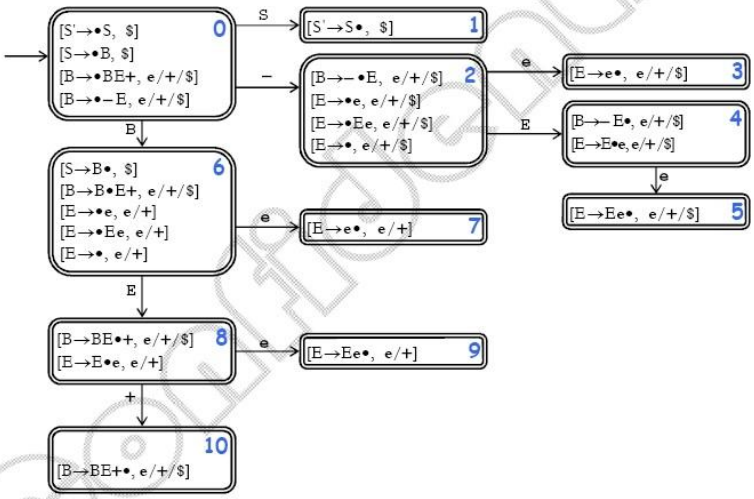
解释：在构造 LR 分析表时拓广文法可以避免文法开始符号出现在产生式右边，从而接受项目是惟一的。

(2) 求 FIRST 集：

FIRST(S) = FIRST(B) = { − }

FIRST(E) = { e , ε }

(3) 构造识别活前缀的 DFA：



(4) 该文法不是一个 LR(1)文法；因为在状态 2（或答状态 4、6）有“移进－归约”冲突，当向前看符号为 e 时产生冲突。

常见错误：

- 未能正确解释拓广文法原因。
- 画 DFA 时向前看符号错。

□

Item #5.2 给定语法制导定义（Syntax-Directed Definition）如下：

产生式	语义动作
S → A B C D	x = 10 * x + 1;
D → **d**	x = x + 5;
C → **c c**	x = 5 * x + 1;
B → B **b**	x = 3 * x + 1;
B → **b**	x = x + 1;
A → **e** B **a**	x = 2 * x + 1;

对于输入句子 **ebbabbccd**，如果在自底向上 LR 分析过程的同时执行语义动作，则上述语义动作将执行 _____ 次；假设 x 的初始值为 0，则语义动作执行完后 x 的最终值为 _____。

【参考答案】8 次（即推导或归约的次数）；x 最终值为 1611。

在 LR 分析过程中执行语义动作的过程如下：

句型	所使用的产生式	计算
S	S → A B C D	10 * 161+ 1 = 1611
ABCD	D → **d**	156 + 5= 161
ABC**d**	C → **c c**	5 * 31 + 1= 156
AB**ccd**	B → B **b**	3 * 10 + 1= 31
AB**bccd**	B → **b**	9 + 1= 10
A**bbccd**	A → **e** B **a**	2 * 4 + 1= 9
eB**abbccd**	B → B **b**	3 * 1 + 1= 4
eB**babbccd**	B → **b**	0 + 1 = 1
ebbabbccd		

Item #5.3 设有如下语言：

```
Expr → for id := int to int do Expr
     | id := Expr
     | Expr ; Expr
     | Expr * Expr
     | Expr + Expr
     | id
     | int
```

定义一个表达式的开销，是其所有子表达式的开销、再加上表达式本身计算的开销。例如，表达式 **a** + E的开销是1加上子表达式 **a** 的开销和子表达式 E 的开销，也可简单地说加法的额外开销是1；类似地，定义乘法的额外开销是2，顺序操作的额外开销是0。赋值的额外开销是1；除额外开销之外，赋值的开销还包括作为右值的子表达式的开销，但不包括作为左值的变量的开销。循环的开销是3再加上循环中每一次迭代的子表达式的开销（即子表达式开销乘以循环次数再加3），每一个变量 **id** 与常量 **int** 作为表达式的开销均为1。

给出一个计算上述开销函数定义的语法制导定义（Syntax-Directed Definition，SDD）。

你可假设有一个属性 val 包含单词的词法分析值；此外，你也可按自己的需要定义其他属性。

Item #5.3【参考答案】注意这一类题目通常存在多种合理的答案。

计算表达式开销的语法制导定义：

语法规则	语义规则
Expr → **for id** := **int**$_1$ **to int**$_2$ **do** Expr$_1$	**if** (int$_2$.val >= int$_1$.val) **then** Expr.cp := (int$_2$.val - int$_1$.val + 1) * Expr$_1$.cp + 3; **else** Expr.cp := 3; **endif**;
Expr → **id** := Expr$_1$	Expr.cp := Expr$_1$.cp + 1;
Expr → Expr$_1$; Expr$_2$	Expr.cp := Expr$_1$.cp + Expr$_2$.cp;
Expr → Expr$_1$ * Expr$_2$	Expr.cp := Expr$_1$.cp + Expr$_2$.cp + 2;
Expr → Expr$_1$ + Expr$_2$	Expr.cp := Expr$_1$.cp + Expr$_2$.cp + 1;
Expr → **id**	Expr.cp := 1;
Expr → **int**	Expr.cp := 1;